

CAP5415 Assignment 1

Austin Robinson

September 2026

1 Process Breakdown

The code implementation follows this process:

- When running the program the user provides a file and standard deviation
- The image is loaded from disk
- A Gaussian and Derivative Gaussian kernel is created using the passed in standard deviation. The default kernel size is 5 by 5
- We take the input image and create two copies. The first copy is smoothed with the Gaussian in the x direction and then we convolve it with the derivative Gaussian kernel also in the x direction. We perform the same steps with the other copy just this time in the y direction
- Then we take both gradient maps and use their values to calculate the gradient magnitude and direction for all pixels.
- Next we pass the gradient and direction maps into the non-max suppression function which performs non max suppression.
- We pass the output of non-max suppression into the hysteresis thresholding function. This implementation auto-generates the thresholds for hysteresis thresholding. We do this by examining the gradient magnitude, we pick the pixel intensity at some percentile to be the high threshold (default for my implementation is 0.9) then for the low threshold we just multiply that percentile by some ratio (default for my implementation is 0.33)
- After this we output the edge detected image.

2 Results

2.1 Results for 3 Sample Images

Below are the result for the 3 images. All of these images were smoothed with $\sigma = 1$ and their high and low thresholds for hysteresis thresholding were determined automatically using intensity values of the gradient map.

Here is a legend for the sub-images:

- a): Image after convolution with X Gaussian
- b): Image after convolution with Y Gaussian
- c): Image after convolution with derivative of X Gaussian
- d): Image after convolution with derivative of Y Gaussian:
- e): Image Gradient map
- f): Final output image.

2.1.1 Image 1

:



Figure 1: Original Image

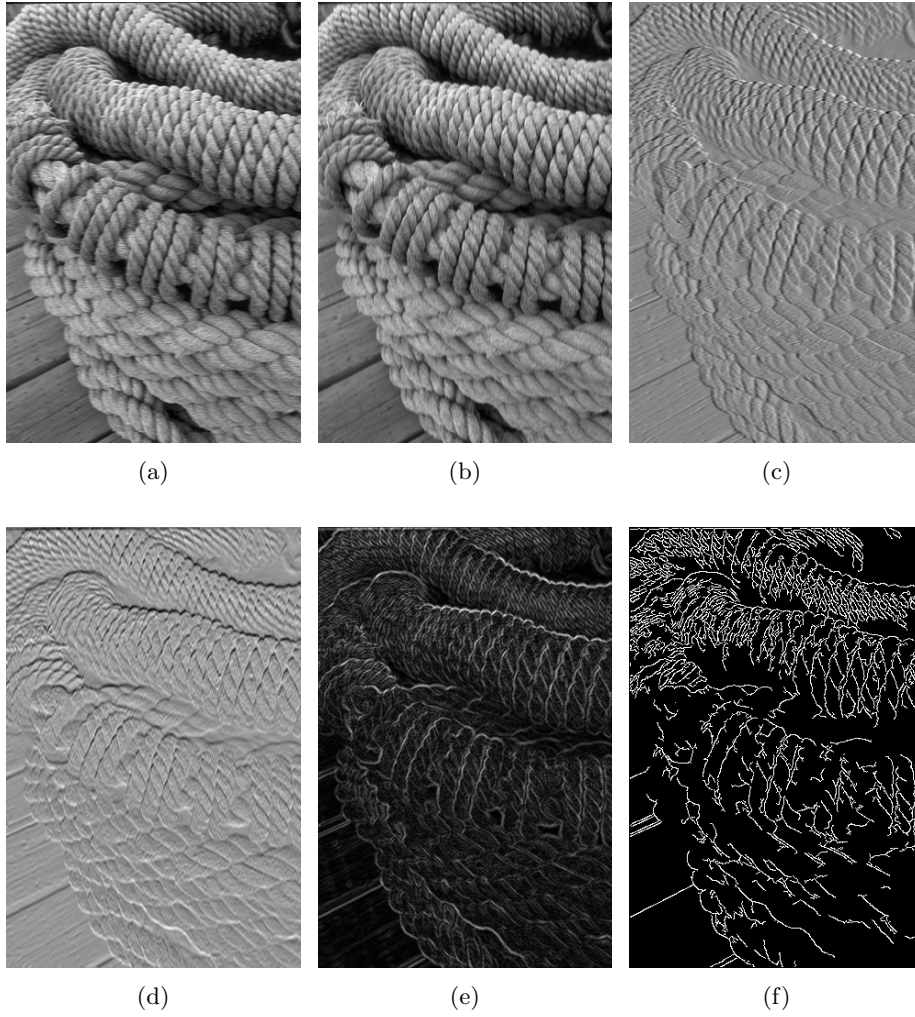


Figure 2: Intermediate and final result for edge detection.

2.1.2 Image 2

:



Figure 3: Original Image

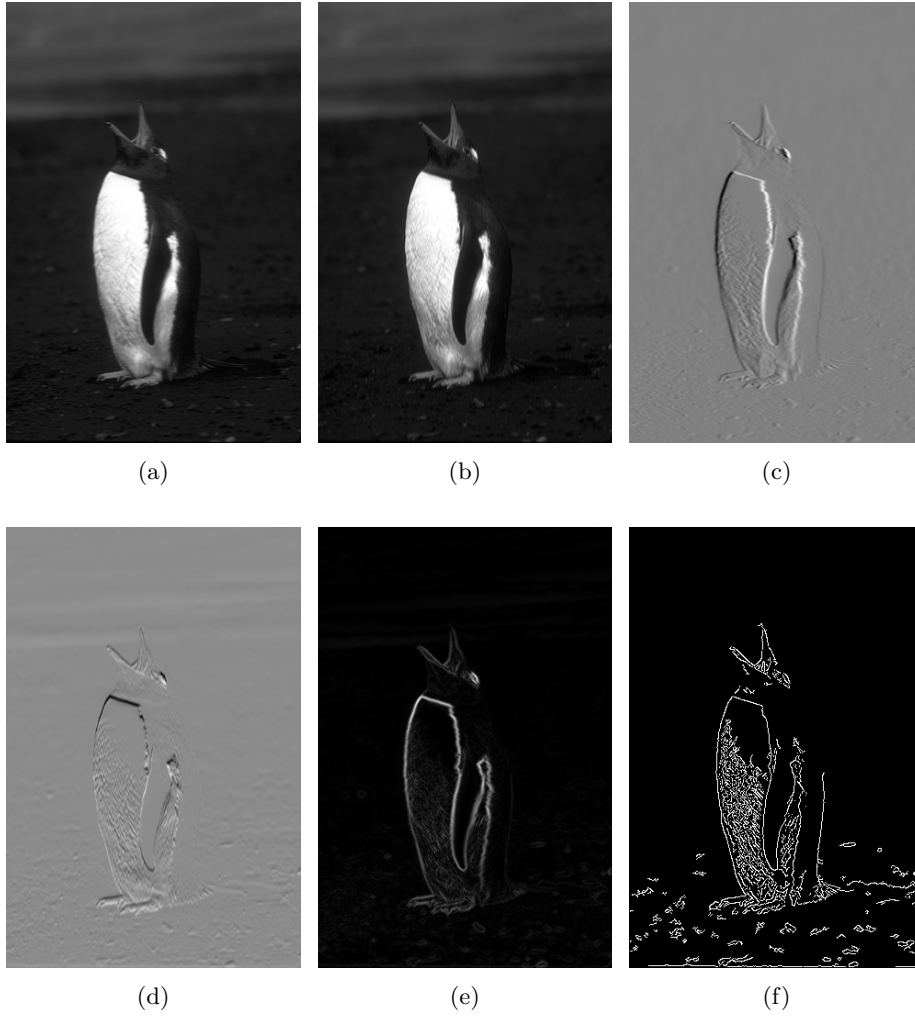


Figure 4: Intermediate and final result for edge detection.

2.1.3 Image 3

:



Figure 5: Original Image

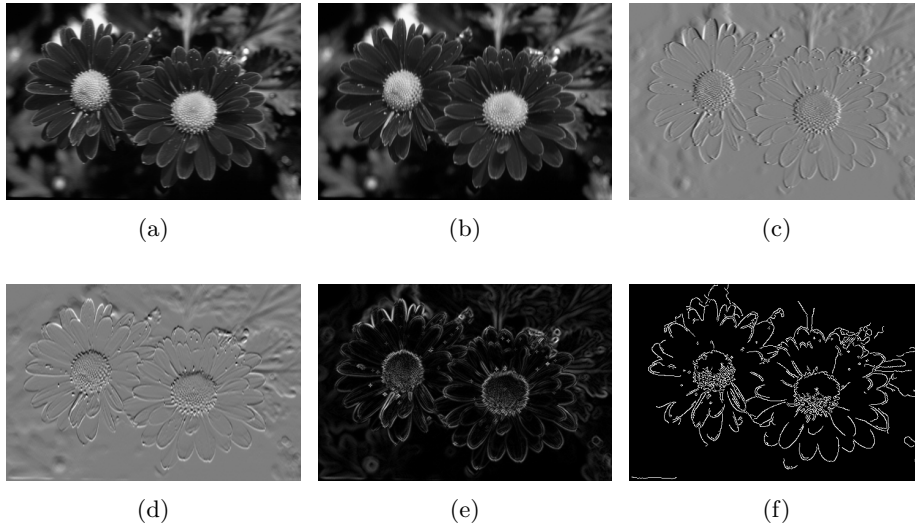


Figure 6: Intermediate and final result for edge detection.

2.2 Effects of Different Standard Deviation

For each of the 3 images I ran the Canny Edge detector for the standard deviation values 0.5, 1 and 5. Here are the results:

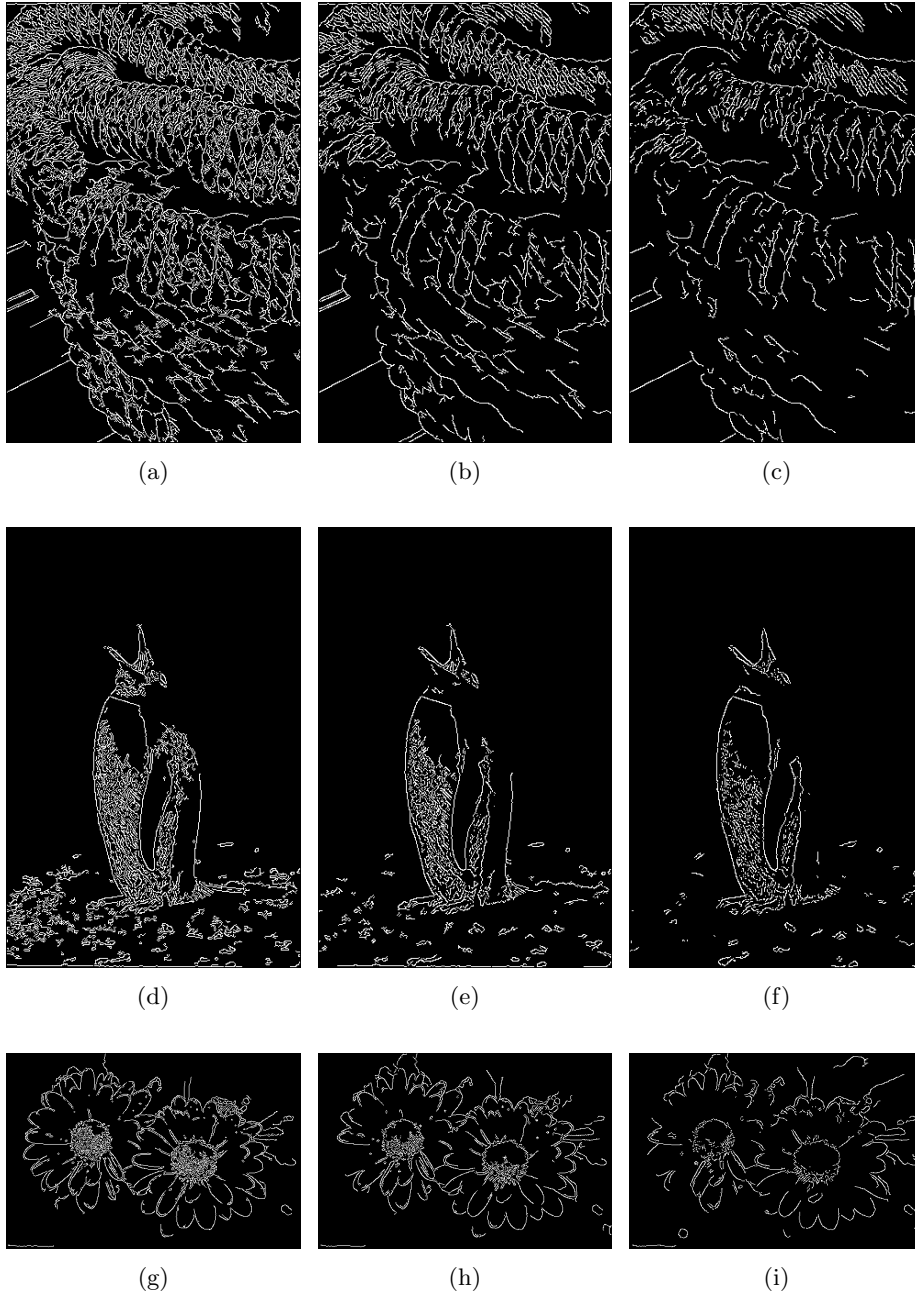


Figure 7: Results with standard deviations of 0.5 (left), 1 (center), 5 (right)

From these results we can identify that as we increase the standard deviation the algorithm starts to detect only the most pronounced edges. With

a standard deviation of 0.5 the algorithm is picking up too much noise in the image, this can really be seen with the penguin image. When the standard deviation is 5 the algorithm is not sensitive enough, this can really be seen with the flower, parts of the main shape of the flower are not detected. Given these 3 standard deviation values I would use the value 1. In my opinion even at 1 it still seems to be a little noisy so I would probably do some testing and maybe something like 2 or 3 would be slightly better. I also think there would be some room to test different kernel sizes to see how that plays a role.